SOFTWARE ENGINEERING LAB – 3

Component Modelling & Architectural Pattern Selection

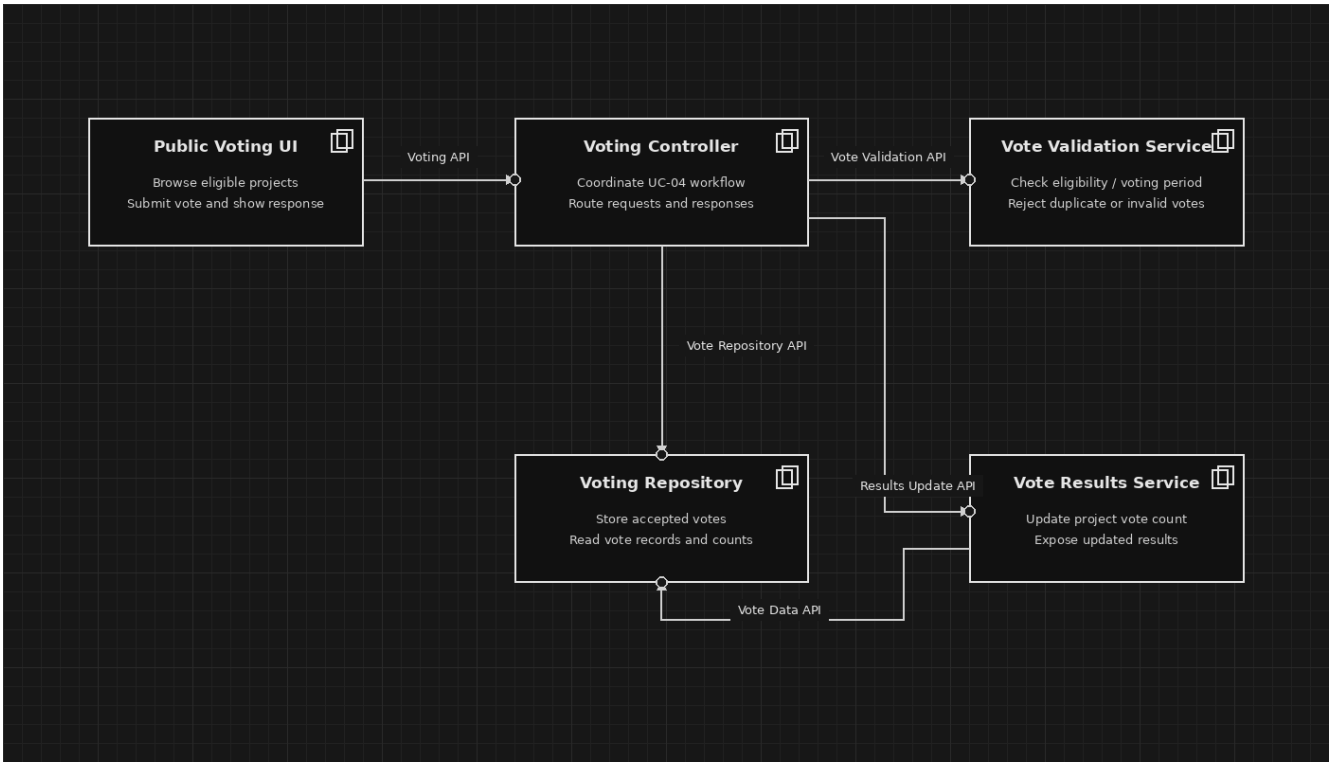
Problem Statement: Student Project Portfolio & Showcase App	**Selected Architectural Style: Layered Architecture**
Student Name: Karanam Sreehitha	**SRN: PES1UG24AM129 | Section: C**

1. Problem Overview

Student Project Portfolio & Showcase App enables student teams to create and present project profiles, share multimedia artifacts, and let panel judges evaluate projects. For the Public Voting use case (UC-04), public participants can browse projects eligible for voting, select a project, and submit a vote. The system must validate each voting request, reject invalid or duplicate attempts, record accepted votes, update the project's vote count, and return a confirmation or rejection message.

2. UML Component Diagram

The component diagram below represents the main modules and labelled interfaces for the Public Voting workflow.



3. Component Responsibilities

- **Public Voting UI:** Allows public participants to browse eligible projects, select a project, submit a vote, and view a confirmation or rejection.
- **Voting Controller:** Coordinates the UC-04 workflow and routes requests between the user interface, validation service, repository, and results service.
- **Vote Validation Service:** Checks project eligibility and voting-period status, detects invalid or duplicate attempts, and returns a validation decision.
- **Voting Repository:** Stores accepted votes, supports checks against existing vote records, and stores project vote counts.
- **Vote Results Service:** Updates the selected project's vote count after an accepted vote and exposes the updated results for the showcase or leaderboard.

4. Architectural Style Selection and Justification

Layered Architecture is suitable because the design separates presentation, workflow coordination, validation, result processing, and data persistence. The Public Voting UI does not directly manipulate stored votes; it sends requests through the Voting Controller. The Vote Validation Service applies the checks required by FR-003 and FR-005 before the Voting Repository records a vote. The Vote Results Service updates the count only after a valid vote is accepted. This separation makes components easier to test and maintain without introducing the overhead of a distributed microservices design for this project's scope.

5. Security and Performance Considerations

Security advantage: The Vote Validation Service centralises eligibility, voting-period, and duplicate/invalid-vote checks. Following FR-003, FR-005 and NFR-001, a rejected request must not be persisted and must not change the project's vote count.

Performance benefit: The request path is kept focused: submit, validate, record, update the count, and return a result. This supports NFR-002, which specifies a confirmation or rejection message within three seconds under normal operating conditions.

6. Conclusion

The proposed component model provides a clear structure for Sreehitha's Public Voting use case. Its five components and labelled interfaces support the required flow from viewing eligible projects through vote validation, persistence, and result updates. The design traces to FR-001 through FR-005 and NFR-001 through NFR-002, while keeping component responsibilities separate and understandable.